단원 06
테이블 분석 패키지
pandas

인공지능소프트웨어학과

강환수 교수

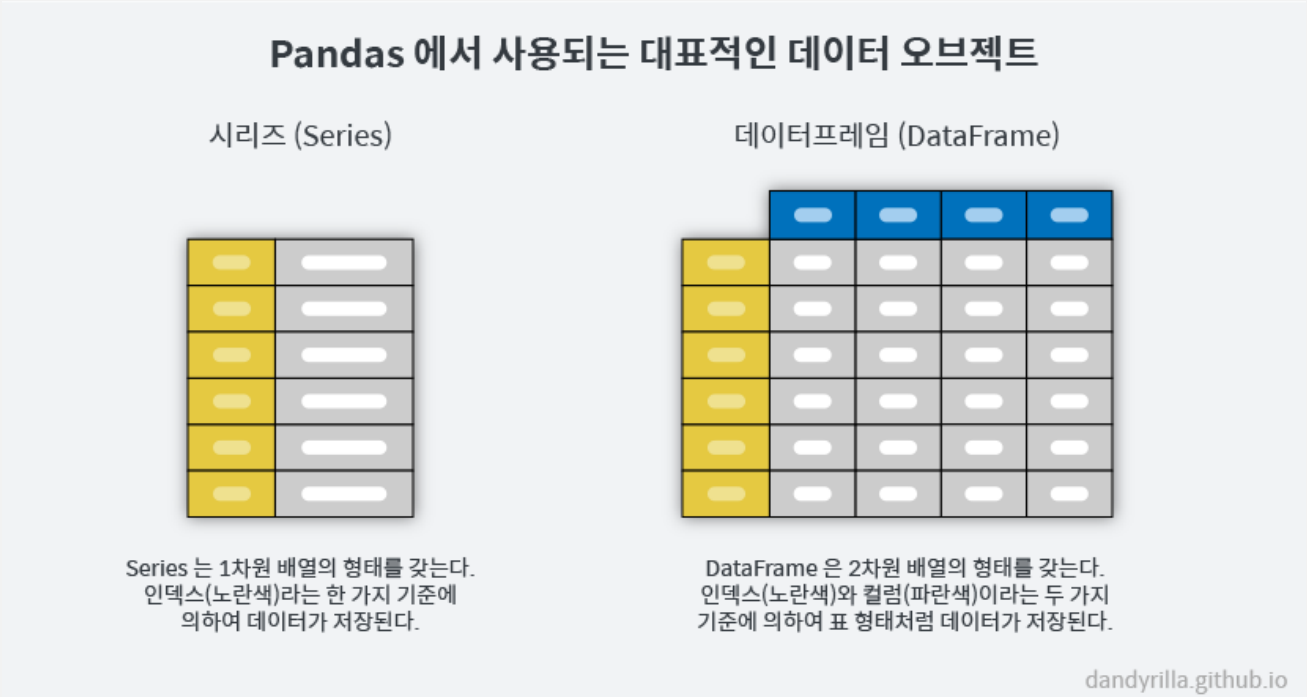
6.1 pandas 개요

-

Pandas 개요

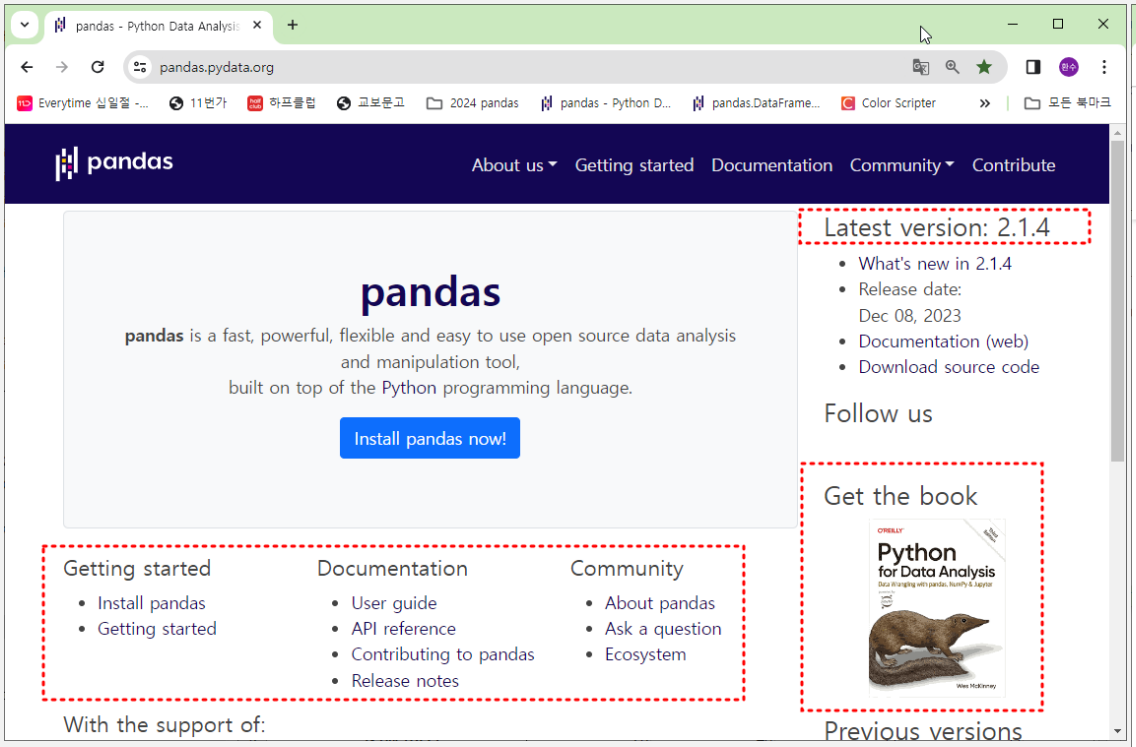
표 형식의 데이터를 처리하고 분석하기 위한 데이터 처리 라이브러리

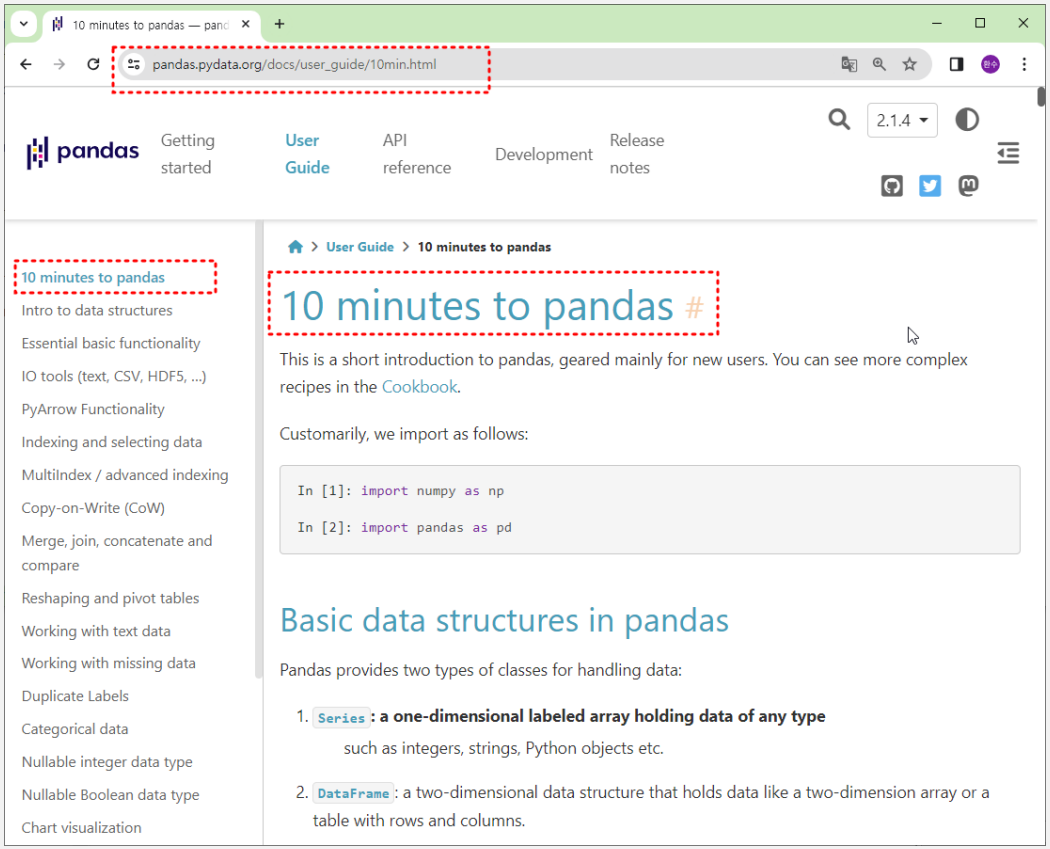
- **주 자료 구조**
  - Series와 DataFrame
    - NumPy를 기반으로 하며, Python, Cython, C 언어로 개발
- **엑셀과 유사한 작업 가능**
  - 데이터를 필터링, 변환, 집계 등 다양한 작업을 수행
  - 대규모 데이터 처리에 유리
    - 엑셀과 유사한 기능을 제공하지만 대용량 데이터를 더 효율적으로 처리



홈 페이지와 ten minutes to pandas

- **ten minutes to pandas**
  - 150여 개의 판다스 코드로 판다스를 이해할 수 있도록 구성

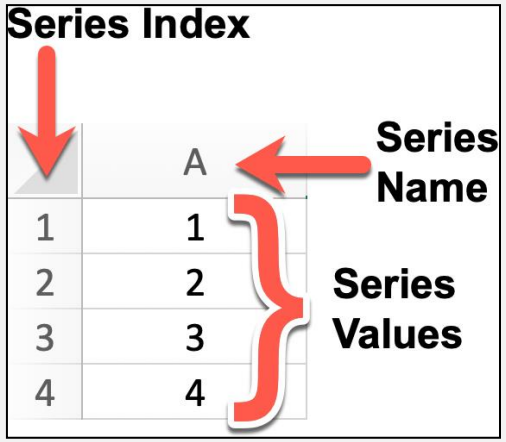


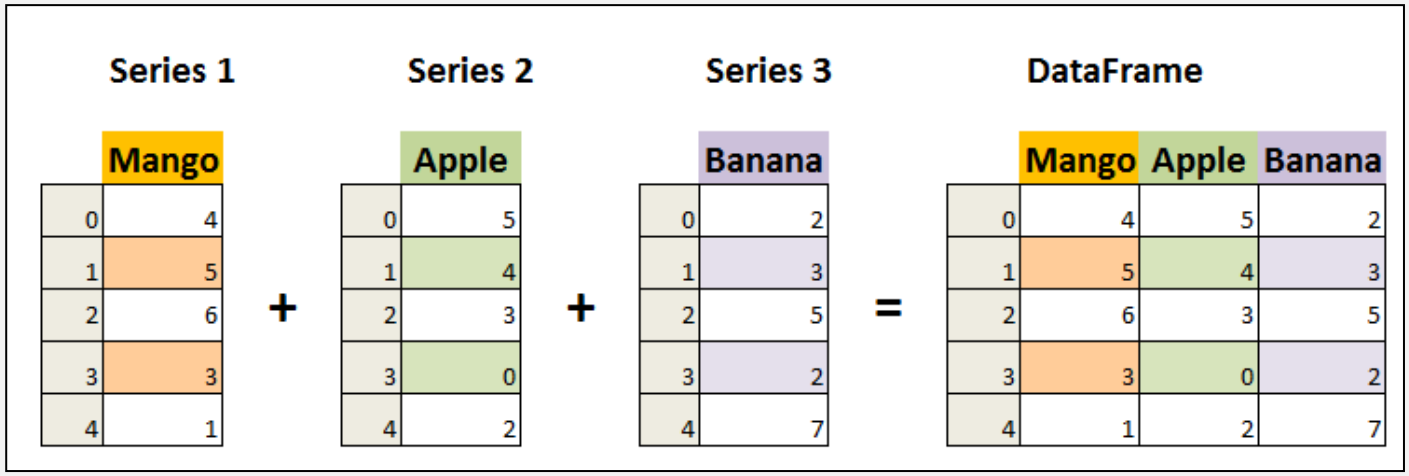


시리즈 개요

values, index, name

- **numpy에서 제공하는 1차원 배열과 비슷**
  - 각 데이터의 의미를 표시하는 인덱스(index)를 붙일 수 있음
  - 데이터 자체는 값(values)
  - 시리즈의 이름인 name 속성도 제공
- **시리즈**
  - 테이블 형태의 데이터프레임(DataFrame)의 열에 해당하는 자료 구조
  - 즉, 여러 시리즈가 모여 데이터프레임이 생성



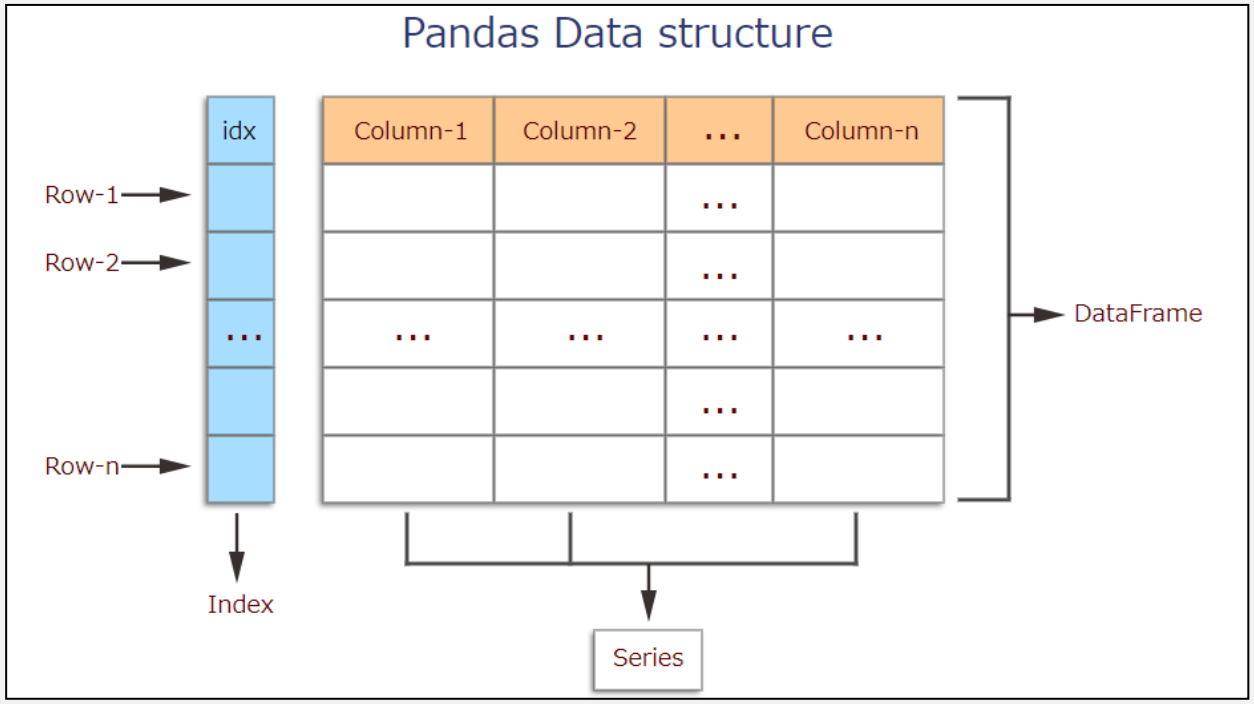


데이터프레임 개요

레이블이 지정된 축 행과 열이 있는 이종 테이블 형식 데이터 구조

- **데이터프레임은 2차원 데이터 구조**
  - 데이터프레임은 데이터, 행(인덱스)과 열(칼럼 라벨)의 세 가지 구성 요소로 구성

  - 데이터프레임은 행과 열에 이름을 붙일 수 있음
    - 따로 붙이지 않으면 RangeIndex라는 0으로 시작하는 시퀀스(sequences)의 정수가 붙음
  - 데이터프레임의 행과 열에 따로 붙이는 이름
    - 행과 열 레이블(label)이라 하며 속성 index와 columns에 저장
  - 행과 열의 레이블이 붙여져도 0부터 시작되는 정수 시퀀스는 사용이 가능



Series 생성

- 키워드 인자 name='first'로 속성 name 지정이 가능

```python
import numpy as np
import pandas as pd

s = pd.Series([1, 3, 5, np.nan, 6, 8], name='first')
s
```

✓ 12.7s

```
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
Name: first, dtype: float64
```

```python
s.name
```

```
'first'
```

```python
s.index
```

```
RangeIndex(start=0, stop=6, step=1)
```

```python
s.values
```

```
array([ 1.,  3.,  5., nan,  6.,  8.])
```

```python
s.shape
```

```
(6,)
```

```python
s.size
```

```
6
```

시리즈 인덱싱과 index, 슬라이싱 1/2

- 레이블 슬라이싱 s['b':'f']
  - b에서 f까지

```
import numpy as np
import pandas as pd

s = pd.Series([1, 3, 5, np.nan, 6, 8], name='first')
s
```

✓ 12.7s

```
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
Name: first, dtype: float64
```

```
s
```

✓ 0.0s

```
0    1.0
1    3.0
2    5.0
3    NaN
4    6.0
5    8.0
Name: first, dtype: float64
```

```
s[1]
```

✓ 0.0s

```
3.0
```

```
s[1:5]
```

✓ 0.0s

```
1    3.0
2    5.0
3    NaN
4    6.0
Name: first, dtype: float64
```

```
s.index = list('abcdef')
s
```

✓ 0.0s

```
a    1.0
b    3.0
c    5.0
d    NaN
e    6.0
f    8.0
Name: first, dtype: float64
```

```
print(s['b'], s['d'])
```

✓ 0.0s

```
3.0 nan
```

```
s['c':'e']
```

✓ 0.0s

```
c    5.0
d    NaN
e    6.0
Name: first, dtype: float64
```

```
s['b':'f']
```

✓ 0.0s

```
b    3.0
c    5.0
d    NaN
e    6.0
f    8.0
Name: first, dtype: float64
```

```
s[0:3]
```

✓ 0.0s

```
a    1.0
b    3.0
c    5.0
Name: first, dtype: float64
```

```
s1 = s[1:4]
s1
```

```
b    3.0
c    5.0
d    NaN
Name: first, dtype: float64
```

```
s1[0]
```

```
3.0
```

```
s1[0] = 10
s1
```

```
a    10.0
b     3.0
c     5.0
Name: first, dtype: float64
```

시리즈 인덱싱과 index, 슬라이싱 2/2

- **새로운 정수 인덱스를 부여**
  - 단일 색인 s[1]
    - 부여된 색인 정수로 참조
- **슬라이싱 색인 s[1:3]**
  - 0부터 시작하는 기본 색인으로 슬라이싱됨
    - 1에서 2까지
  - 즉, 새로 지정한 정수 라벨로 슬라이싱되지 않음

```
s
```
✓ 0.0s

```
a     1.0
b    10.0
c     5.0
d     NaN
e     6.0
f     8.0
Name: first, dtype: float64
```

```
s.index = np.arange(1, 7)
s
```
✓ 0.0s

```
1     1.0
2    10.0
3     5.0
4     NaN
5     6.0
6     8.0
Name: first, dtype: float64
```

```
s[1]
```
✓ 0.0s

```
1.0
```

```
s[0]
```
⊗ 1.0s

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File d:\ProgramData\anaconda3_240601\Li
   3804 try:
-> 3805     return self._engine.get_loc
   3806 except KeyError as err:

File index.pyx:167, in pandas._libs.ind

File index.pyx:196, in pandas._libs.ind

File pandas\\_libs\\hashtable_class_hel

File pandas\\_libs\\hashtable_class_hel

KeyError: 0

The above exception was the direct caus

KeyError                                  Traceback (most recent call last)
Cell In[69], line 1
----> 1 s[0]

File d:\ProgramData\anaconda3_240601\Li
   1118     return self._values[key]
...
   3815     #  InvalidIndexError. Other
   3816     #  the TypeError.
   3817     self._check_indexing_error(

KeyError: 0
```

Output is truncated. View as a scrollable element

```
s[1:3]
```
✓ 0.0s

```
2    10.0
3     5.0
Name: first, dtype: float64
```

시리즈 고급 인덱싱

- df[[첨자_목록]]으로 각 원소로 구성된 시리즈를 반환

```
import pandas as pd

age = pd.Series([9, 17, 32, 45, 65, 98], index=list('abcdef'), name='age')
age
```

✓ 0.0s

```
a     9
b    17
c    32
d    45
e    65
f    98
Name: age, dtype: int64
```

```
age['b']
```

✓ 0.0s

```
17
```

```
age[[1, 3, 5]]
```

✓ 0.0s

```
C:\Users\PC\AppData\Local\T
  age[[1, 3, 5]]
```

```
b    17
d    45
f    98
Name: age, dtype: int64
```

```
age[['a', 'f', 'd']]
```

✓ 0.0s

```
a     9
f    98
d    45
Name: age, dtype: int64
```

```
age[['c']]
```

✓ 0.0s

```
c    32
Name: age, dtype: int64
```

시리즈 논리 참조(불리안 인덱싱)

- **조건 s > 0**
  - 논리(boolean) 결과의 시리즈를 반환
- **s[s > 0]**
  - 조건 연산을 만족하는 값으로 구성된 시리즈를 반환

```python
import pandas as pd

s = pd.Series([3, 6, -3, 0, -4, 8, -7])
s
```

```
0    3
1    6
2   -3
3    0
4   -4
5    8
6   -7
dtype: int64
```

```python
s > 0
```

✓ 0.0s

```
0     True
1     True
2    False
3    False
4    False
5     True
6    False
dtype: bool
```

```python
s%2 == 0
```

✓ 0.0s

```
0    False
1     True
2    False
3     True
4     True
5     True
6    False
dtype: bool
```

```python
s[s > 0]
```

✓ 0.0s

```
0    3
1    6
5    8
dtype: int64
```

```python
s[~(s > 0)]
```

✓ 0.0s

```
2   -3
3    0
4   -4
6   -7
dtype: int64
```

```python
s[s%2 == 0]
```

```
1    6
3    0
4   -4
5    8
dtype: int64
```

```python
s[s%2 != 0]
```

```
0     3
1    17
2    -3
5     5
6    -7
dtype: int64
```

```python
s[(s > 0) & (s%2 == 0)]
```

```
1    6
5    8
dtype: int64
```

Series의 정렬

- **시리즈의 name과 index의 name도 지정 가능**
  - 시리즈의 출력에서도 시리즈의 name과 index의 name도 표시
- **함수 sort_values()**
  - 시리즈의 값을 오름차순으로 정렬
- **함수 sort_values(ascending=False)**
  - 시리즈의 값을 내림차순으로 정렬
- **함수 sort_index()**
  - 학생이름인 index의 값을 오름차순으로 정렬
- **함수 sort_index(ascending=False)**
  - index의 값을 내림차순으로 정렬

```python
import numpy as np
import pandas as pd

s = ['희빈', '수영', '현수', '지호', '지민']
p = pd.Series([99, 84, 92, 65, 78], index=s)
p
```

✓ 0.0s

```
희빈    99
수영    84
현수    92
지호    65
지민    78
dtype: int64
```

```python
p.name = '수학성적'
p.index.name = 'st_name'
p
```

✓ 0.0s

```
st_name
희빈    99
수영    84
현수    92
지호    65
지민    78
Name: 수학성적, dtype: int64
```

```python
p.sort_values()
```

✓ 0.0s

```
st_name
지호    65
지민    78
수영    84
현수    92
희빈    99
Name: 수학성적, dtype: int64
```

```python
p.sort_values(ascending=False)
```

✓ 0.0s

```
st_name
희빈    99
현수    92
수영    84
지민    78
지호    65
Name: 수학성적, dtype: int64
```

```python
p.sort_index()
```

```
st_name
수영    84
지민    78
지호    65
현수    92
희빈    99
Name: 수학성적, dtype: int64
```

```python
p.sort_index(ascending=False)
```

```
st_name
희빈    99
현수    92
지호    65
지민    78
수영    84
Name: 수학성적, dtype: int64
```

시리즈의 기본 통계 함수

- **함수 mean(), median()**
  - 시리즈의 평균값과 중간값
- **함수 max(), min()**
  - 시리즈의 최댓값과 최솟값
- **함수 count()와 속성 size**
  - 시리즈의 원소 수
- **describe()**
  - 다양한 통계 값 일괄 표시

```python
import numpy as np
import pandas as pd

s = ['희빈', '수영', '현수', '지호', '지민']
p = pd.Series([99, 84, 92, 65, 78], index=s)
p
```

✓ 0.0s

```
희빈    99
수영    84
현수    92
지호    65
지민    78
dtype: int64
```

```python
print(p.max())
print(p.min())
```

```
99
65
```

```python
print(p.mean())
print(p.median())
```

```
83.6
84.0
```

```python
print(p.count())
print(p.size)
```

```
5
5
```

```python
p.describe()
```

```
count     5.000000
mean     83.600000
std      13.088163
min      65.000000
25%      78.000000
50%      84.000000
75%      92.000000
max      99.000000
Name: 수학성적, dtype: float64
```

메소드 count(), value_counts()

np.nan이나 None: 값이 없는 결측값(missing value)

- 결측값은 NaN(Not a Number)이나 nan으로 표시
- 함수 s.count()
  - 결측값을 제외한 값의 시리즈의 원소 수가 반환
- 함수 s.value_counts()
  - 고유한 값의 개수를 포함하는 시리즈를 반환
    - 가장 자주 나온 값을 내림차순으로 정렬
    - 기본적으로 결측 값인 NA 값을 제외
    - 키워드 인자 dropna=False
      - 결측값인 NA 값을 포함

```python
import numpy as np
import pandas as pd

s = pd.Series([3, 1, 1, 2, None, np.nan, 4, 6, 6, np.nan])
s
```

✓ 0.0s

```
0    3.0
1    1.0
2    1.0
3    2.0
4    NaN
5    NaN
6    4.0
7    6.0
8    6.0
9    NaN
dtype: float64
```

```python
s.values
```

```
array([ 3.,  1.,  1.,  2., nan, nan,  4.,  6.,  6., nan])
```

```python
s.count()
```

```
7
```

```python
s.value_counts()
```

```
1.0    2
6.0    2
3.0    1
2.0    1
4.0    1
dtype: int64
```

```python
s.value_counts(dropna=False)
```

```
NaN    3
1.0    2
6.0    2
3.0    1
2.0    1
4.0    1
dtype: int64
```

s.value_counts(bins=4)

- 키워드 인자 bins=4로 각 값의 빈도를 구하는 구간 수를 지정
  - 결과 시리즈의 첨자가 구간
    - 값이 해당 구간의 빈도수
  - 많은 빈도 구간부터 정렬되어 표시
  - 구간의 왼쪽은 미포함이며 오른쪽은 포함
    - 반 열린 구간(half-open bins) 방식

```python
import numpy as np
import pandas as pd

s = pd.Series([3, 1, 1, 2, None, np.nan, 4, 6, 6, np.nan])
s
```

✓ 0.0s

```
0    3.0
1    1.0
2    1.0
3    2.0
4    NaN
5    NaN
6    4.0
7    6.0
8    6.0
9    NaN
dtype: float64
```

```python
s.value_counts(sort=False, dropna=False)
```

```
3.0    1
1.0    2
2.0    1
NaN    3
4.0    1
6.0    2
dtype: int64
```

```python
s.value_counts(bins=4)
```

```
(0.994, 2.25]    3
(4.75, 6.0]      2
(2.25, 3.5]      1
(3.5, 4.75]      1
dtype: int64
```

```python
s.value_counts(bins=4, sort=False)
```

```
(0.994, 2.25]    3
(2.25, 3.5]      1
(3.5, 4.75]      1
(4.75, 6.0]      2
dtype: int64
```

```python
s.value_counts(bins=[0.5, 3.5, 6.5], sort=False)
```

```
(0.499, 3.5]    4
(3.5, 6.5]      3
dtype: int64
```

6.2 데이터프레임 생성

-

2차원 행렬의 테이블 데이터 사용

- **첫 인자에 1차원 리스트를 사용**
  - 하나의 열이 있는 데이터프레임을 생성
  - 행과 열의 레이블은 기본인 RangeIndex()인 정수 시퀀스

```python
import pandas as pd

df = pd.DataFrame([10, 20, 30, 40, 50])
df
```

	0
0	10
1	20
2	30
3	40
4	50

```python
import pandas as pd

df = pd.DataFrame([[1, 2, 3], [10, 20, 30], [100, 200, 300]], index=list('abc'))
df
```

	0	1	2
a	1	2	3
b	10	20	30
c	100	200	300

```python
df = pd.DataFrame([[1, 2, 3], [10, 20, 30], [100, 200, 300]],
                  index=list('abc'), columns=['A', 'B', 'C'])
df
```

	A	B	C
a	1	2	3
b	10	20	30
c	100	200	300

```python
print(df.index)
print(df.columns)
```

```
Index(['a', 'b', 'c'], dtype='object')
Index(['A', 'B', 'C'], dtype='object')
```

2차원 행렬의 테이블 데이터 사용

- **첫 인자에 1차원 리스트를 사용**
  - 하나의 열이 있는 데이터프레임을 생성
  - 행과 열의 레이블은 기본인 RangeIndex()인 정수 시퀀스

```python
import numpy as np
import pandas as pd

df1 = pd.DataFrame(data = np.arange(12).reshape(3, 4))
df1
```

	0	1	2	3
0	0	1	2	3
1	4	5	6	7
2	8	9	10	11

```python
import numpy as np
import pandas as pd

df1 = pd.DataFrame(np.random.randint(1, 46, (5, 6)))
df1
```

	0	1	2	3	4	5
0	36	10	31	17	1	37
1	4	18	29	2	26	7
2	36	29	38	45	30	13
3	39	35	36	14	9	34
4	34	2	25	8	32	30

```python
df1 = pd.DataFrame(np.random.random((5, 3)))
df1
```

	0	1	2
0	0.069106	0.840559	0.586908
1	0.161549	0.433819	0.437954
2	0.016900	0.096814	0.802116
3	0.597695	0.748465	0.647906
4	0.698881	0.399377	0.719364

dict의 테이블 데이터 사용

- **dict 형태의 자료를 사용**
  - 키가 데이터프레임 열의 제목인 레이블
  - 열은 입력한 순서대로

```
df = pd.DataFrame({'국어': [82, 76, 67], '수학': [99, 87, 78]},
                  index=['김국진', '신영희', '이길순'])
df
```

	국어	수학
김국진	82	99
신영희	76	87
이길순	67	78

```
df.index
```

```
Index(['김국진', '신영희', '이길순'], dtype='object')
```

```
df.columns
```

```
Index(['국어', '수학'], dtype='object')
```

```
import pandas as pd

df = pd.DataFrame({'numbers': [1, 2, 3],
                   'colors': ['red', 'white', 'blue']})
df
```

	numbers	colors
0	1	red
1	2	white
2	3	blue

```
df.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

```
df.columns
```

```
Index(['numbers', 'colors'], dtype='object')
```

여러 Series로 DataFrame 생성

```python
courses = pd.Series(["Spark","PySpark","Hadoop"], name='courses')
courses
```

```
0      Spark
1    PySpark
2     Hadoop
Name: courses, dtype: object
```

```python
fees = pd.Series([250000, 300000, 200000], name='fees')
fees
```

```
0    250000
1    300000
2    200000
Name: fees, dtype: int64
```

```python
df = pd.DataFrame({courses.name: courses, fees.name: fees})
df
```

	courses	fees
0	Spark	250000
1	PySpark	300000
2	Hadoop	200000

```python
courses = pd.Series(["Spark","PySpark","Hadoop"], name='courses')
fees = pd.Series([250000, 300000, 200000], name='fees')

df = pd.concat([courses,fees], axis=1)
df
```

	courses	fees
0	Spark	250000
1	PySpark	300000
2	Hadoop	200000

```python
courses = pd.Series(["Spark","PySpark","Hadoop"], name='courses')
fees = pd.Series([250000, 300000, 200000], name='fees')

df = pd.concat([courses,fees])
df
```

```
0      Spark
1    PySpark
2     Hadoop
0     250000
1     300000
2     200000
dtype: object
```

다양한 데이터프레임 생성 방법

- **하나의 시리즈가 행이 됨**
  - 시리즈의 name은 행의 index가 됨

```
courses = pd.Series(["Spark","PySpark","Hadoop"], name='courses')
fees = pd.Series([250000, 300000, 200000], name='fees')

df = pd.DataFrame([courses,fees])
df
```

	0	1	2
courses	Spark	PySpark	Hadoop
fees	250000	300000	200000

```
df.T
```

	courses	fees
0	Spark	250000
1	PySpark	300000
2	Hadoop	200000

```
df = pd.DataFrame()
df
```

```
courses = pd.Series(["Spark","PySpark","Hadoop"], name='courses')
df[courses.name] = courses
df
```

	courses
0	Spark
1	PySpark
2	Hadoop

```
fees = pd.Series([250000, 300000, 200000], name='fees')
df[fees.name] = fees
df
```

	courses	fees
0	Spark	250000
1	PySpark	300000
2	Hadoop	200000

여러 dict의 목록 데이터 사용

- **사전의 목록**
  - [dict1, dict2, ...]
- **하나의 dict 항목이 하나의 행에 해당**
  - 키에 해당하는 값이 없으면 결측값인 NaN이 입력

```python
import pandas as pd

df= pd.DataFrame([{'국어': 70, '수학': 89, '영어': 90},
                  {'국어': 80, '영어': 99},
                  {'수학': 66, '영어': 78}])
df
```

	국어	수학	영어
0	70.0	89.0	90
1	80.0	NaN	99
2	NaN	66.0	78

```python
import pandas as pd

s1 = pd.Series([90, 80, 60, 66])
s2 = pd.Series([45, 73, 83, 99])
s3 = pd.Series([92, 75, 82, 75])
s1
```

```
0    90
1    80
2    60
3    66
dtype: int64
```

```python
df = pd.DataFrame({'국어': s1, '영어': s2, '수학': s3})
df
```

	국어	영어	수학
0	90	45	92
1	80	73	75
2	60	83	82
3	66	99	75

```python
df.index = ['노', '장', '박', '강']
df
```

	국어	영어	수학
노	90	45	92
장	80	73	75
박	60	83	82
강	66	99	75

데이터프레임의 결측값 표시 NaN

Not A Number

- 시리즈의 인덱스가 바로 데이터프레임의 행 레이블이 됨
  - 시리즈의 인덱스와 같은 데이터프레임의 인덱스에 값이 저장
  - 지정되지 않은 원소는 결측값인 NaN(Not a Number)으로 저장

```
df = pd.DataFrame({'A': pd.Series([10, 20, 30], index=[0, 1, 2]),
                   'B': pd.Series([3, 2, 1], index=[1, 2, 3])})
df
```

	A	B
0	10.0	NaN
1	20.0	3.0
2	30.0	2.0
3	NaN	1.0

- 시리즈 2개를 index를 붙여 생성하고, 1개는 기본 인덱스로 생성
  - 시리즈를 사용하고 index도 지정해서 df를 생성
    - 결과적으로 데이터프레임의 행 레이블에 맞는 시리즈 인덱스의 자료만이 저장

```
s1 = pd.Series([90, 80, 60], index=['노', '장', '박'])
s2 = pd.Series([45, 73, 99], index=['장', '박', '강'])
s3 = pd.Series([92, 75, 82, 75])
```

```
df = pd.DataFrame({'국어': s1, '영어': s2, '수학': s3},
                  index=['노', '장', '박', '강'])
df
```

	국어	영어	수학
노	90.0	NaN	NaN
장	80.0	45.0	NaN
박	60.0	73.0	NaN
강	NaN	99.0	NaN

df.dtypes

- **자료형**
  - 각 열 자료형

```
d = {'col1': [1, 2], 'col2': [3, 4]}
df = pd.DataFrame(data=d)
df
```

	col1	col2
0	1	3
1	2	4

```
df.dtypes
```

```
col1    int64
col2    int64
dtype: object
```

```
df = pd.DataFrame(data=d, dtype=np.int8)
df.dtypes
```

```
col1    int8
col2    int8
dtype: object
```

dtypes: object는 "이 결과가 문자열을 담은 Series"라는 뜻이고, 실제 데이터 타입 정보는 위의 열 이름들에서 확인

여러 날짜 시퀀스를 위한 함수 date_range()

- **함수 date_range()**
  - 연속된 날짜인 DatetimeIndex 생성
  - 2025년 1월 1일부터 6일간의 시퀀스를 생성
    - 위 반환 값의 원소는 시간 정보를 나타내는 Timestamp 자료형

```
dates = pd.date_range("20250101", periods=6)
dates
```

```
DatetimeIndex(['2025-01-01', '2025-01-02', '2025-01-03', '2025-01-04',
               '2025-01-05', '2025-01-06'],
              dtype='datetime64[ns]', freq='D')
```

```
dates[0]
```

```
Timestamp('2025-01-01 00:00:00', freq='D')
```

- **인자 periods가 없다면 end 인자로 지정**
  - 인자 end는 포함되며 마지막 값

```
dates = pd.date_range("20250101", end='20250110')
dates
```

```
DatetimeIndex(['2025-01-01', '2025-01-02', '2025-01-03', '2025-01-04',
               '2025-01-05', '2025-01-06', '2025-01-07', '2025-01-08',
               '2025-01-09', '2025-01-10'],
              dtype='datetime64[ns]', freq='D')
```

인자 freq='3D' 등

- 인자 freq='3D'
  - 다음 시퀀스를 3일 후로 지정
- 인자 freq='3H'
  - 다음 시퀀스를 3시간 후로 지정
- 인자 freq='W'
  - 다음 시퀀스를 1주 후로 지정
  - 시작일은 start에 지정한 날을 포함해서 이후 첫 일요일이 시작
    - 키워드 인자 freq='W-SUN'

```
dates = pd.date_range("20250101", periods=6, freq='3D')
dates
```

```
DatetimeIndex(['2025-01-01', '2025-01-04', '2025-01-07', '2025-01-10',
               '2025-01-13', '2025-01-16'],
              dtype='datetime64[ns]', freq='3D')
```

```
dates = pd.date_range("20250101", periods=6, freq='3H')
dates
```

```
DatetimeIndex(['2025-01-01 00:00:00', '2025-01-01 03:00:00',
               '2025-01-01 06:00:00', '2025-01-01 09:00:00',
               '2025-01-01 12:00:00', '2025-01-01 15:00:00'],
              dtype='datetime64[ns]', freq='3H')
```

```
dates = pd.date_range("20250101", periods=6, freq='W')
dates
```

✓ 0.0s

```
DatetimeIndex(['2025-01-05', '2025-01-12', '2025-01-19', '2025-01-26',
               '2025-02-02', '2025-02-09'],
              dtype='datetime64[ns]', freq='W-SUN')
```

```
dates = pd.date_range("20260101", periods=6, freq='W-SUN')
dates
```

✓ 0.0s

```
DatetimeIndex(['2026-01-04', '2026-01-11', '2026-01-18', '2026-01-25',
               '2026-02-01', '2026-02-08'],
              dtype='datetime64[ns]', freq='W-SUN')
```

인자 freq='W-TUE' 등

- 키워드 인자 freq='W-TUE'
  - 시작일이 매주 화요일
- 키워드 인자 freq='W-FRI'
  - 시작일이 매주 금요일
- 인자 freq='M'로 다음 시퀀스를 1달 후로 지정(deprecated)
  - 시작일은 start에 지정한 날을 포함해서 월의 마지막 일자가 시작
  - freq='ME'로 지정하세요!
- 인자 freq='MS'로 지정
  - 시작일은 start에 지정한 날을 포함해서 월의 마지막 시작 일자가 됨

```python
dates = pd.date_range("20260101", periods=6, freq='W-TUE')
dates
```

✓ 0.0s

```
DatetimeIndex(['2026-01-06', '2026-01-13', '2026-01-20', '2026-01-27',
               '2026-02-03', '2026-02-10'],
              dtype='datetime64[ns]', freq='W-TUE')
```

```python
dates = pd.date_range("20260101", periods=6, freq='W-FRI')
dates
```

✓ 0.0s

```
DatetimeIndex(['2026-01-02', '2026-01-09', '2026-01-16', '2026-01-23',
               '2026-01-30', '2026-02-06'],
              dtype='datetime64[ns]', freq='W-FRI')
```

```python
dates = pd.date_range("20250101", periods=6, freq='M')
dates
```

```
DatetimeIndex(['2025-01-31', '2025-02-28', '2025-03-31', '2025-04-30',
               '2025-05-31', '2025-06-30'],
              dtype='datetime64[ns]', freq='M')
```

```python
dates = pd.date_range("20250101", periods=6, freq='MS')
dates
```

```
DatetimeIndex(['2025-01-01', '2025-02-01', '2025-03-01', '2025-04-01',
               '2025-05-01', '2025-06-01'],
              dtype='datetime64[ns]', freq='MS')
```

날짜를 포함한 데이터프레임

- 변수 dates에는 연속된 날짜 6개가 저장
  - 자동으로 freq='D'
- 위 dates를 index로 지정
  - 열 명은 각각 A, B, C, D로 지정한 df 생성

```
dates = pd.date_range("20250302", periods=6)
dates
```

```
DatetimeIndex(['2025-03-02', '2025-03-03', '2025-03-04', '2025-03-05',
               '2025-03-06', '2025-03-07'],
              dtype='datetime64[ns]', freq='D')
```

```
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
df
```

	A	B	C	D
2025-03-02	-0.979014	0.888473	-0.691063	1.905381
2025-03-03	-1.652958	-0.049652	1.960006	1.193716
2025-03-04	0.161388	-0.031089	0.166527	0.819051
2025-03-05	0.803170	-0.129082	-0.242391	0.668687
2025-03-06	-0.662178	-0.433686	1.185859	-0.924045
2025-03-07	1.317815	-0.162820	1.982848	0.904207

다양한 자료형으로 구성된 데이터프레임

- **데이터프레임 df2**
  - 열 명이 'A'에서 'F'
  - 행 명이 0에서 3

```python
df2 = pd.DataFrame({
        "A": 1.0,
        "B": pd.Timestamp("20260302"),
        "C": pd.Series(np.arange(1, 3, .5)),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo"
    })
df2
```

✓ 0.0s

	A	B	C	D	E	F
0	1.0	2026-03-02	1.0	3	test	foo
1	1.0	2026-03-02	1.5	3	train	foo
2	1.0	2026-03-02	2.0	3	test	foo
3	1.0	2026-03-02	2.5	3	train	foo

```python
df2.dtypes
```

✓ 0.0s

```
A          float64
B    datetime64[s]
C          float64
D            int32
E         category
F           object
dtype: object
```

6.3 DataFrame의 다양한 참조

-

데이터프레임 열 참조

- **열로 구성된 시리즈 반환**
  - df[‘열명’]
  - df.열명
- **열로 구성된 데이터프레임 반환**
  - df[[‘열명’]]
  - df[[‘열명1’, ‘열명2’, ‘열명3’ …]]

```python
import pandas as pd

pf = pd.DataFrame(data = [[25, 35, 8, 18],
                          [18, 27, 10, 20],
                          [17, 17, 10, 19],
                          [12, 22, 9, 20],
                          [22, 34, 8, 16]],
                  index=['윤일형', '강수희', '홍소희', '유한빈', '신수빈'],
                  columns=['중간', '기말', '과제', '출석'])
pf
```

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```python
pf['중간']
```

```
윤일형    25
강수희    18
홍소희    17
유한빈    12
신수빈    22
Name: 중간, dtype: int64
```

```python
pf.중간
```

```
윤일형    25
강수희    18
홍소희    17
유한빈    12
신수빈    22
Name: 중간, dtype: int64
```

```python
pf['출석']
```

```
윤일형    18
강수희    20
홍소희    19
유한빈    20
신수빈    16
Name: 출석, dtype: int64
```

```python
pf[['출석']]
```

✓ 0.0s

	출석
윤일형	18
강수희	20
홍소희	19
유한빈	20
신수빈	16

```python
pf[['기말', '출석', '중간']]
```

	기말	출석	중간
윤일형	35	18	25
강수희	27	20	18
홍소희	17	19	17
유한빈	22	20	12
신수빈	34	16	22

```python
pf[['기말']]
```

	기말
윤일형	35
강수희	27
홍소희	17
유한빈	22
신수빈	34

조건에 맞는 행 참조

- **df[조건]**

```python
import pandas as pd

pf = pd.DataFrame(data = [[25, 35, 8, 18],
                          [18, 27, 10, 20],
                          [17, 17, 10, 19],
                          [12, 22, 9, 20],
                          [22, 34, 8, 16]],
                  index=['윤일형', '강수희', '홍소희', '유한빈', '신수빈'],
                  columns=['중간', '기말', '과제', '출석'])
pf
```

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```python
pf['중간'] > 20
```

```
윤일형     True
강수희    False
홍소희    False
유한빈    False
신수빈     True
Name: 중간, dtype: bool
```

```python
pf[pf['중간'] > 20]
```

	중간	기말	과제	출석
윤일형	25	35	8	18
신수빈	22	34	8	16

```python
pf[pf['출석'] >= 19]
```

	중간	기말	과제	출석
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20

```python
pf[(pf['출석'] >= 19) & (pf['기말'] >= 20)]
```

	중간	기말	과제	출석
강수희	18	27	10	20
유한빈	12	22	9	20

df.loc[]

레이블이나 부울 배열을 통해 행과 열 그룹에 참조

- **조건**
  - df.loc[조건]
- **단일 레이블**
  - df.loc[1], df.loc['a']
- **레이블 목록 또는 배열**
  - df.loc[['a', 'b', 'c']
- **레이블 슬라이스**
  - df.loc['a':'c']

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```python
import pandas as pd

pf = pd.DataFrame(data = [[25, 35, 8, 18],
                          [18, 27, 10, 20],
                          [17, 17, 10, 19],
                          [12, 22, 9, 20],
                          [22, 34, 8, 16]],
                  index=['윤일형', '강수희', '홍소희', '유한빈', '신수빈'],
                  columns=['중간', '기말', '과제', '출석'])
pf
```

```python
pf.loc['윤일형']
```

```
중간    25
기말    35
과제     8
출석    18
Name: 윤일형, dtype: int64
```

```python
pf.loc[['윤일형']]
```

	중간	기말	과제	출석
윤일형	25	35	8	18

```python
pf.loc[['홍소희', '윤일형']]
```

	중간	기말	과제	출석
홍소희	17	17	10	19
윤일형	25	35	8	18

```python
pf.loc['윤일형':'유한빈']
```

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20

```python
pf.loc[pf.중간 > 20]
```

	중간	기말	과제	출석
윤일형	25	35	8	18
신수빈	22	34	8	16

```python
pf.loc[(pf.중간 > 10) & (pf.과제 == 10)]
```

	중간	기말	과제	출석
강수희	18	27	10	20
홍소희	17	17	10	19

loc[index[i]]로 행 참조

- **pf.loc[pf.index[0]]으로 첫 번째의 행 결과를 시리즈로 반환**
  - pf.index[0]는 첫 번째 행 이름을 반환

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```python
import pandas as pd

pf = pd.DataFrame(data = [[25, 35, 8, 18],
                          [18, 27, 10, 20],
                          [17, 17, 10, 19],
                          [12, 22, 9, 20],
                          [22, 34, 8, 16]],
                  index=['윤일형', '강수희', '홍소희', '유한빈', '신수빈'],
                  columns=['중간', '기말', '과제', '출석'])
pf
```

```python
pf.loc[pf.index[0]] # 첫 번째 행 검색
```

```
중간    25
기말    35
과제     8
출석    18
Name: 윤일형, dtype: int64
```

```python
pf.loc[[pf.index[0]]] # 첫 번째 행 검색
```

	중간	기말	과제	출석
윤일형	25	35	8	18

```python
pf.loc[pf.index[1:4]] # 슬라이싱 행 검색
```

	중간	기말	과제	출석
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20

```python
pf.loc[pf.index[1:4]][['기말', '과제']]
```

	기말	과제
강수희	27	10
홍소희	17	10
유한빈	22	9

열 참조

- pf.loc[:, '열명']으로 '열명' 전체를 시리즈로 반환

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```python
import pandas as pd

pf = pd.DataFrame(data = [[25, 35, 8, 18],
                          [18, 27, 10, 20],
                          [17, 17, 10, 19],
                          [12, 22, 9, 20],
                          [22, 34, 8, 16]],
                  index=['윤일형', '강수희', '홍소희', '유한빈', '신수빈'],
                  columns=['중간', '기말', '과제', '출석'])
pf
```

```python
pf.loc[:, '과제']
```

```
윤일형     8
강수희    10
홍소희    10
유한빈     9
신수빈     8
Name: 과제, dtype: int64
```

```python
pf.loc[:, '중간':'과제']
```

	중간	기말	과제
윤일형	25	35	8
강수희	18	27	10
홍소희	17	17	10
유한빈	12	22	9
신수빈	22	34	8

```python
pf.loc[:, ['기말', '출석']]
```

	기말	출석
윤일형	35	18
강수희	27	20
홍소희	17	19
유한빈	22	20
신수빈	34	16

loc[:, columns[i]]로 열 참조

- **df.loc[:, pf.columns[0]]으로 첫 번째 열 결과를 시리즈로 반환**
  - df.columns[0]은 첫 번째 열 이름을 반환

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.loc[:, pf.columns[0]] # 
```

```
윤일형    25
강수희    18
홍소희    17
유한빈    12
신수빈    22
Name: 중간, dtype: int64
```

```
pf.loc[:, [pf.columns[0]] ]
```

	중간
윤일형	25
강수희	18
홍소희	17
유한빈	12
신수빈	22

```
pf.loc[:, pf.columns[1:]] #
```

	기말	과제	출석
윤일형	35	8	18
강수희	27	10	20
홍소희	17	10	19
유한빈	22	9	20
신수빈	34	8	16

```
pf.loc[:, pf.columns[1:3]]
```

✓ 0.0s

	기말	과제
윤일형	35	8
강수희	27	10
홍소희	17	10
유한빈	22	9
신수빈	34	8

```
pf.loc[:, pf.columns[1:]][['기말', '과제']]
```

	기말	과제
윤일형	35	8
강수희	27	10
홍소희	17	10
유한빈	22	9
신수빈	34	8

행과 열 참조

- df.loc['행명', '열명']으로 행과 열에 맞는 한 원소를 참조

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.loc['윤일형', '출석']
```

```
18
```

```
pf.loc['윤일형':'유한빈', '기말']
```

```
윤일형    35
강수희    27
홍소희    17
유한빈    22
Name: 기말, dtype: int64
```

```
pf.loc['윤일형':'유한빈', '기말':'과제']
```

	기말	과제
윤일형	35	8
강수희	27	10
홍소희	17	10
유한빈	22	9

```
pf.loc[pf.index[0], pf.columns[1]]
```

```
35
```

```
pf.loc[pf.index[0:3], pf.columns[2]]
```

```
윤일형     8
강수희    10
홍소희    10
Name: 과제, dtype: int64
```

```
pf.loc[pf.index[0:3], pf.columns[2:]]
```

	과제	출석
윤일형	8	18
강수희	10	20
홍소희	10	19

df.iloc[정수] 참조

- **행 참조**
  - df.iloc[정수]로 정수 index의 행을 참조

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.iloc[1]
```

```
중간    18
기말    27
과제    10
출석    20
Name: 강수희, dtype: int64
```

```
pf.iloc[[1]]
```

	중간	기말	과제	출석
강수희	18	27	10	20

```
pf.iloc[[1, 3]]
```

	중간	기말	과제	출석
강수희	18	27	10	20
유한빈	12	22	9	20

```
pf.iloc[2:]
```

	중간	기말	과제	출석
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

df.iloc[:, 정수] 참조

- **열 참조**
  - df.iloc[:, 정수]로 정수 columns의 열을 참조

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.iloc[:, 2]
```

```
윤일형     8
강수희    10
홍소희    10
유한빈     9
신수빈     8
Name: 과제, dtype: int64
```

```
pf.iloc[:, [2]]
```

	과제
윤일형	8
강수희	10
홍소희	10
유한빈	9
신수빈	8

```
pf.iloc[:, 1:]
```

	기말	과제	출석
윤일형	35	8	18
강수희	27	10	20
홍소희	17	10	19
유한빈	22	9	20
신수빈	34	8	16

df.iloc[행번호, 열번호] 참조

- 행과 열에 대당하는 원소나 시리즈, 데이터프레임 참조열 참조
  - df.iloc[행번호, 열번호]

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.iloc[1, 1]
```

```
27
```

```
pf.iloc[1:3, 1:]
```

	기말	과제	출석
강수희	27	10	20
홍소희	17	10	19

```
pf.iloc[:, 1:]
```

	기말	과제	출석
윤일형	35	8	18
강수희	27	10	20
홍소희	17	10	19
유한빈	22	9	20
신수빈	34	8	16

```
pf.iloc[[1, 3, 4], 1:]
```

	기말	과제	출석
강수희	27	10	20
유한빈	22	9	20
신수빈	34	8	16

```
pf.iloc[[1, 3, 4], [0, 1]]
```

	중간	기말
강수희	18	27
유한빈	12	22
신수빈	22	34

at['행명', '열명'] 참조

- 레이블 기반으로 참조하는 방식으로 단 하나의 원소를 참조
- 하나의 값을 참조하므로 속도가 빠른 장점

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.at['윤일형', '과제']
```

```
8
```

```
pf.at['유한빈', '기말']
```

```
22
```

```
pf.at[pf.index[1], '출석']
```

```
20
```

```
pf.at[pf.index[3], pf.columns[2]]
```

```
9
```

iat[i, j] 참조

- 정수 첨자 기반의 참조 방식으로 단 하나의 원소를 참조

	중간	기말	과제	출석
윤일형	25	35	8	18
강수희	18	27	10	20
홍소희	17	17	10	19
유한빈	12	22	9	20
신수빈	22	34	8	16

```
pf.iat[1, 2]
```

```
10
```

```
pf.iat[2, 3]
```

```
19
```